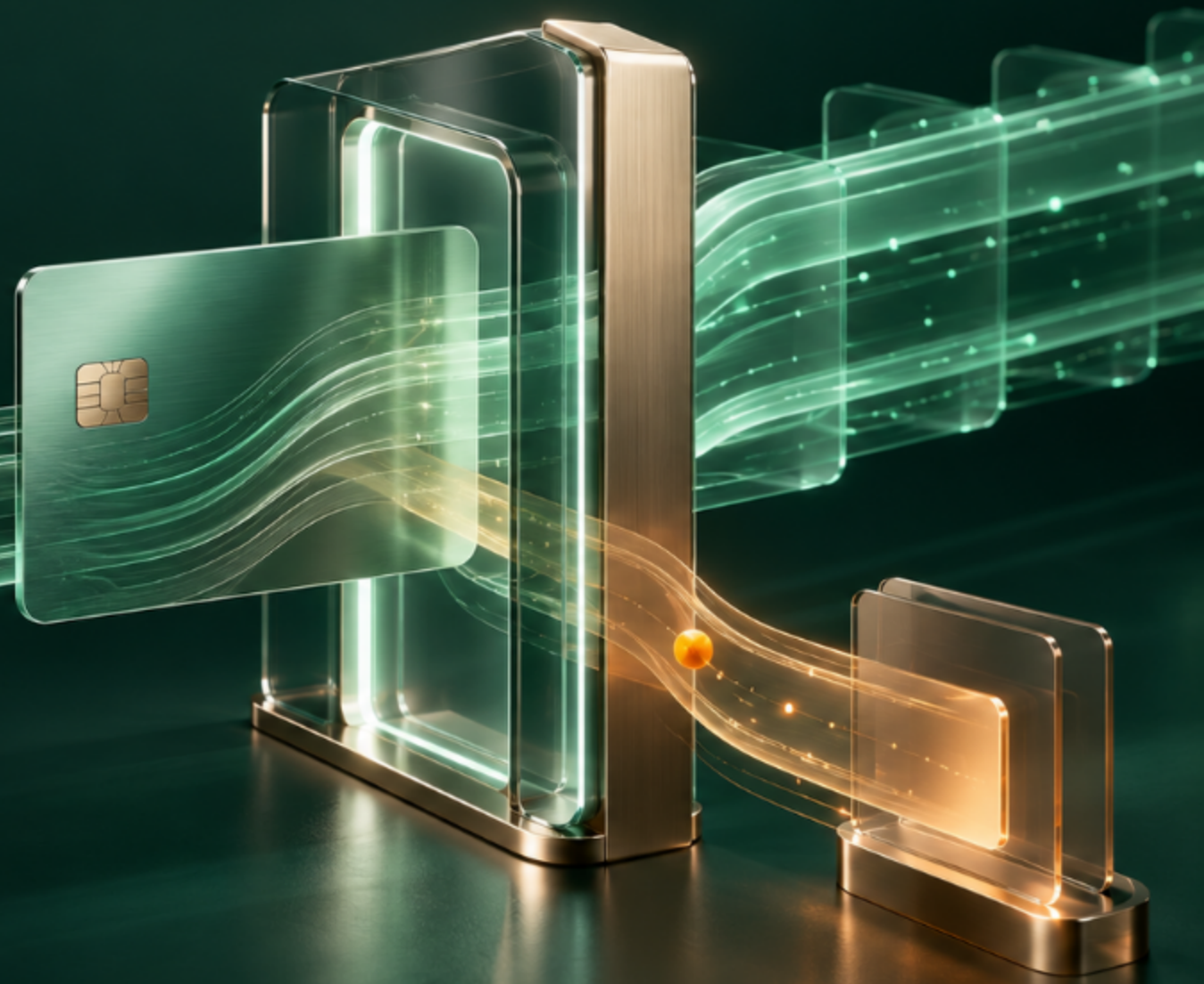
Signal platform

DeliveryGuard and Fraud Guard pilot review

Architecture, synthetic demonstrations and acceptance gates

Decision requested:
approve a scoped sandbox evaluation

29 October 2026



The pilot decision

Approve a bounded internal evaluation

One sandbox workspace. A synthetic dataset per pilot workstream. Named engineering and operations owners.

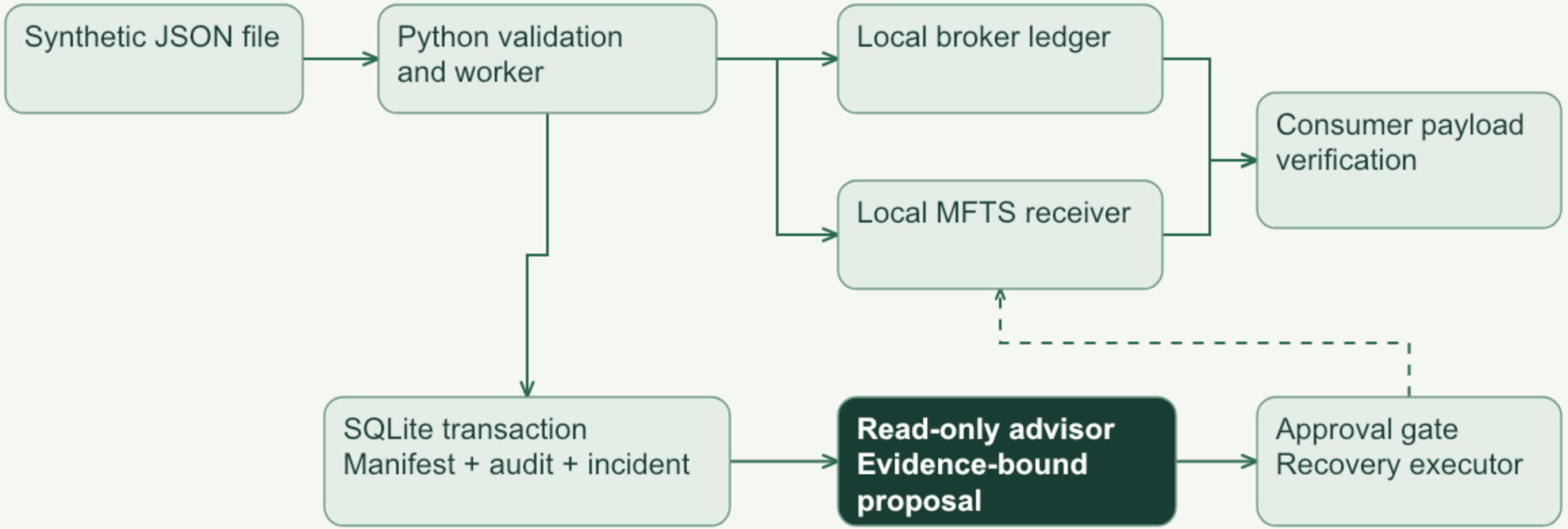
Business question

Can operators recover the failed delivery channel while preserving successful delivery evidence?

Boundary of acceptance

Architecture direction and pilot scope. Production approval follows separate security and reliability gates.

Current executable architecture



Flask portal exposes workflow status, Signal investigations and lineage from the same persisted run

Real implementation and synthetic boundaries

Area	Real in this repository	Synthetic or pending
Processing	File reads, validation, routing, hashing	Synthetic records and virtual clock
Delivery	Adapter calls, manifests and retries	Local broker and receiver, no network
Investigation	Deterministic risk rules and lineage	Declared consumer dependencies
Recovery	Evidence checks and selective retry	Simulated requester and approver
Platform	Bundle generation and connector code	No Databricks execution verified

The local execution profile runs Delivery Lab Python operations. Exported Spark programs require a Databricks runtime.

Guided demo walkthrough

Step	Manager sees	Evidence to inspect
1. Workflow ready	Generated plan and task graph	Plan fingerprint and frozen configuration
2. Execute	Validation passes, MFTS fails	5 broker records, 0 transfer artifacts
3–4. Detect and investigate	BREACHED risk and exposed reports	Channel manifest, lineage and reasons
5–6. Request and approve	Request then separate approval	Action bound to current evidence hash
7–8. Recover and verify	MFTS completes, reports turn green	Kafka stays at 5, payload checks pass

Live entry: http://127.0.0.1:8080/demo
Autoplay takes about one minute

Recovery proof in the golden scenario

Observed evidence	After failure	After recovery and verification
Accepted Kafka records	5	5
MFTS artifacts	0	5
Verified consumer reports	0	2
Workflow status	Recovery required	SUCCEEDED
Approval state	Not requested	EXECUTED

Recovery preserves the missed deadline in history. Green reports require content verification.

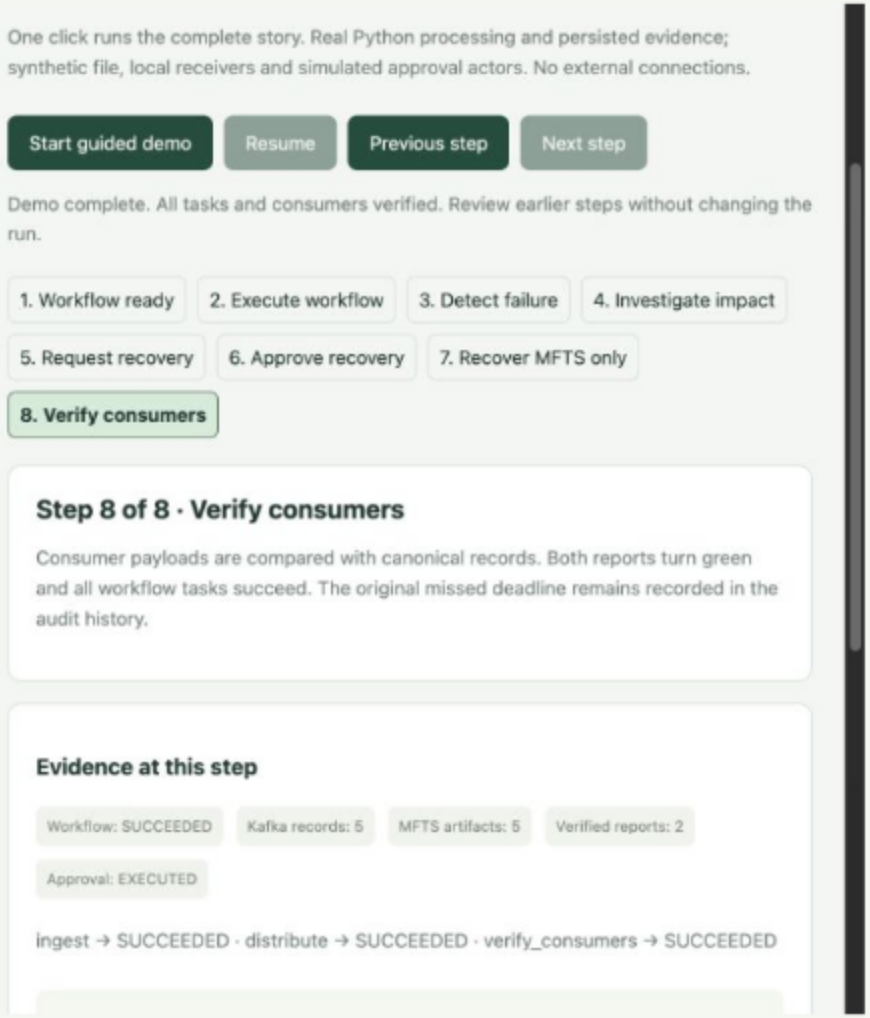
Guided demo evidence

A manager can inspect each decision

Pause at failure to inspect the red channel and affected reports.

Revisit saved steps after recovery without changing the underlying run.

Expand fingerprints and audit details when the architect asks for proof.



Controls that make recovery reviewable

Stable identities	Per-record distribution IDs and payload fingerprints separate identical duplicates from conflicting content.
Ambiguous acknowledgments	UNKNOWN delivery requires reconciliation before retry. A timeout does not prove non-delivery.
Evidence-bound approval	Execution rejects missing or stale approval. Requester and approver must differ in the local protocol.
Verified completion	Consumer checks compare receiver payloads with canonical records before reports become healthy.

DeliveryGuard baseline evidence

09

Python tests passed

9

Evaluated fault scenarios

5

Kafka records before and after recovery

Covered locally

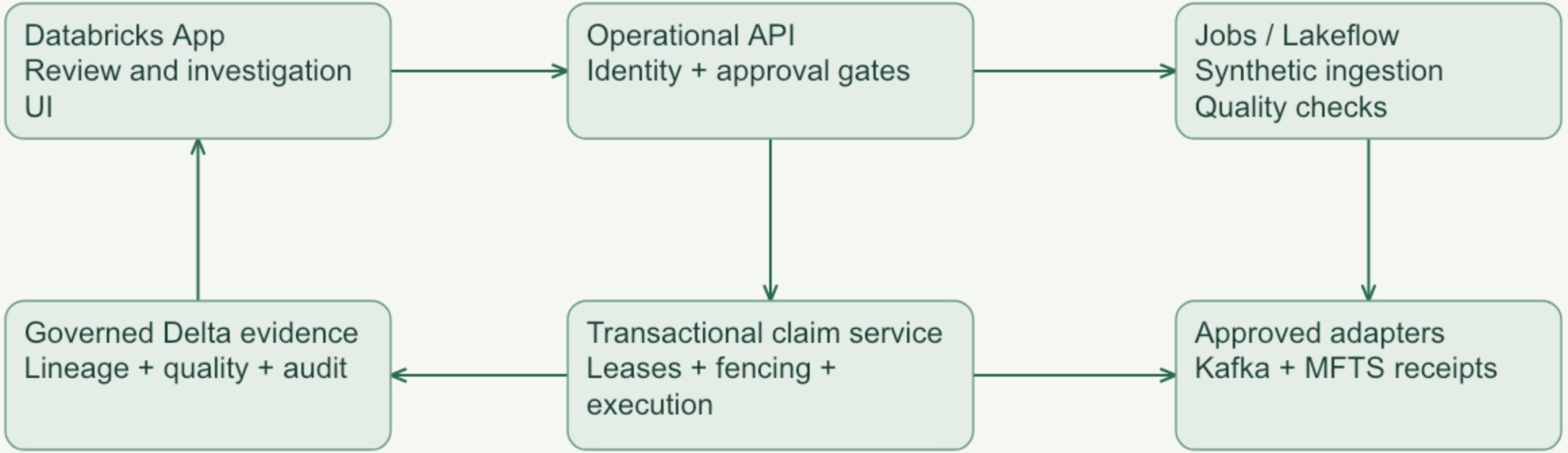
Timeout reconciliation, retry exhaustion, quarantine, duplicate suppression, stale approval and transaction rollback.

Still unmeasured

Production throughput, live connector behavior, enterprise authorization and operator triage improvement.

Proposed internal Databricks architecture

Proposed deployment topology, subject to sandbox validation



Advisory reads stay separate from authorized execution. Operational ownership requires a proven claim protocol.

Pilot entry gates

Gate and owner	Acceptance evidence	Current position
Scope / architect	One isolated workspace, dataset and named owner	Decision requested
Deployment / platform	Bundle validates and App + synthetic Job run successfully	Generated config only
Access / security	Least-privilege grants and agreed per-user visibility	Design and tests required
Identity / application team	Server-enforced roles, distinct approver and expiry	Local simulated actors only
Data / data owner	Approved synthetic fixtures and retention policy	Fixtures available

Pilot exit criteria

Gate	Pass condition	Evidence and owner
Failure and recovery	Injected failures detect correctly and preserve acknowledged writes	Scenario results / engineering
Approvals	Unauthorized, expired and stale actions fail server-side	Negative tests / security
Restart and concurrency	No double claim or lost evidence across agreed crash windows	Restart and contention tests / platform
Consumer verification	Receipts and payload checks gate completion	Reconciliation results / data owner
Operations and value	Agreed alerting, cost cap and measured triage baseline	Runbook + study / operations

Any unresolved safety or access-control failure blocks expansion to live endpoints.

DeliveryGuard pilot sequence

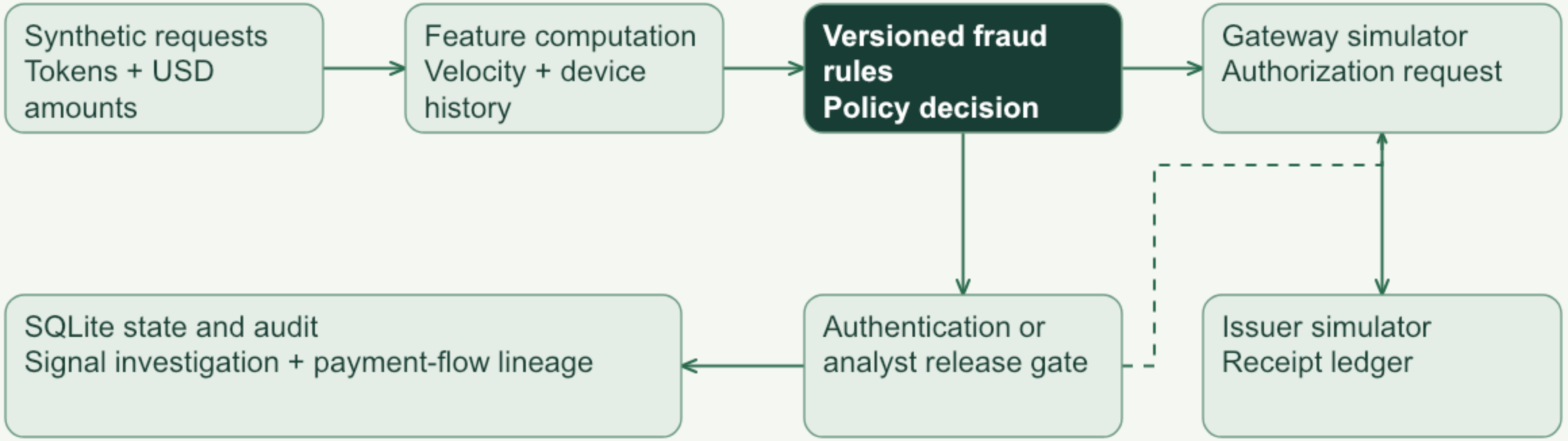
Phase	Deliverable	Decision
1. Architecture review	Contracts, identity model and ownership	Accept bounded design direction
2. Synthetic Databricks pilot	App, jobs, governed evidence and negative tests	Confirm entry and exit gates
3. Approved adapter validation	Kafka/MFTS receipts and crash recovery evidence	Authorize limited endpoint access
4. Operator evaluation	Triage baseline, measured value and support model	Decide whether to expand

Requested today

Sponsor the sandbox pilot, nominate accountable owners and record acceptance conditions.

Fraud Guard architecture

Implemented local payment path with merchant-side screening



Only eligible transactions reach authorization. A local issuer receipt resolves uncertain gateway outcomes.

Fraud, authentication and payment decisions

Decision	Owner in the MVP	Guardrail
Fraud clearance	Deterministic rules	ALLOW, CHALLENGE, REVIEW, BLOCK or UNAVAILABLE
Customer authentication	Local challenge simulator	Pass before expiry before proceeding
Analyst release	Simulated analyst and supervisor	REVIEW only, fresh evidence and expiry
Payment authorization	Gateway and issuer simulators	Issuer may decline after fraud clearance
Unknown authorization	Receipt reconciliation	No blind retry or duplicate authorization

An analyst release never overrides a block, failed authentication or an issuer decline.

Fraud Guard: real versus synthetic

Area	Real implementation	Synthetic or pending
Input and features	Strict contract and rolling attempt counts	SYN tokens, generated history, virtual clock
Fraud checks	Versioned rules and reason codes	Illustrative thresholds, no trained model
Payments	Idempotency, state transitions and receipts	Local gateway and issuer adapters
Approvals	Distinct roles, evidence hash and expiry checks	Simulated names, no enterprise identity
Investigation	Persisted Signal case and historical snapshots	Declared flow, no live card or table lineage

No real card data, payment network, EMV 3DS, Databricks or model connection participates.

Guided payment demo

Nine steps, about one minute

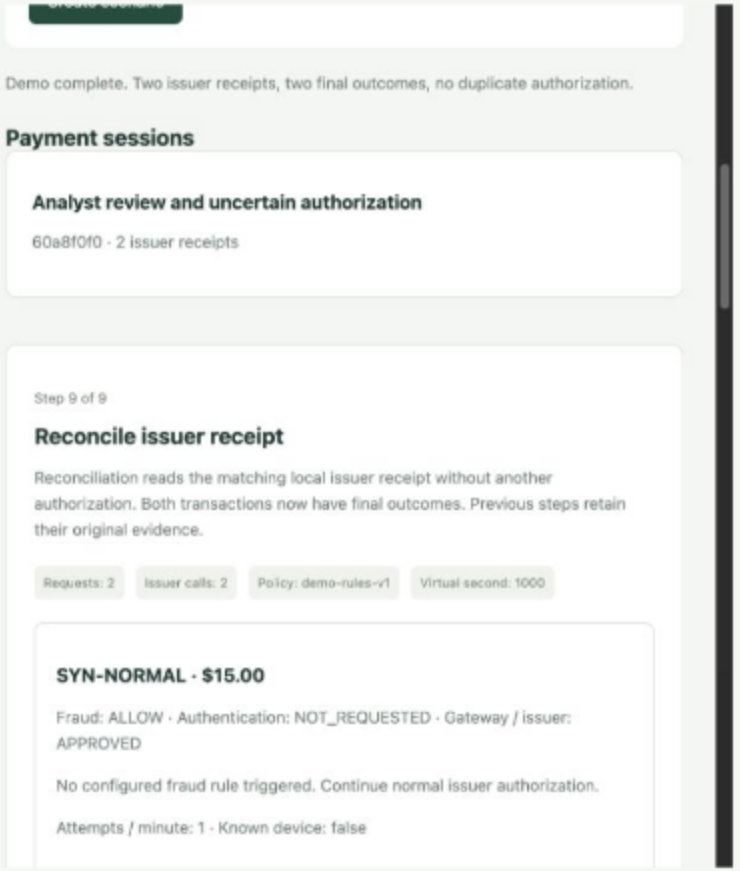
1. Receive two synthetic payment requests

2–3. Screen both, authorize the normal purchase

4. Investigate the flagged purchase in Signal

5–7. Request, approve and execute release

8–9. Reconcile an issuer acknowledgment timeout



Observed result: 2 requests, 2 issuer receipts

Live entry: http://127.0.0.1:8080/fraud

Databricks pilot integration for Fraud Guard

Layer	Proposed pilot responsibility	Required evidence
Databricks data plane	Governed synthetic events, features and evaluation jobs	Schema contracts, freshness and replay results
Online decision service	Rules, current features and bounded responses	Measured latency, deadline and outage behavior
Operational state	Tenant-scoped idempotency and durable claims	Concurrency, restart and crash-window tests
Databricks App / Signal	Authenticated investigation and approval surface	Role enforcement and visibility tests
Payment adapters	Simulator first, approved test endpoints later	Provider-specific receipts and reconciliation

Current Fraud Guard runs locally. Databricks deployment and Workflow Builder integration remain pending.

Fraud Guard pilot acceptance criteria

Gate	Pass condition	Evidence owner
Decision separation	Blocks, failed authentication and outages make zero issuer calls	Payments engineering
Authorization safety	Duplicates reuse results. UNKNOWN requires reconciliation	Payments + platform
Approval and authentication	Missing, wrong-role, stale and expired approvals fail	Security + application
Databricks deployment	Synthetic job runs, feature freshness and restart checks pass	Databricks platform
Value and operations	Service meets agreed latency and cost limits. Outage checks pass	Risk + operations

Local evidence: 59 platform tests, including 15 Fraud Guard tests, and 8 synthetic scenario evaluations.

Requested acceptance and next decisions

Accept a synthetic Databricks pilot with explicit conditions

Architecture sponsor	Confirm merchant-side placement and scope
Payments and risk owners	Approve decision policy, release limits and outage behavior
Platform and security owners	Select state, identity and evidence controls
Pilot review	Evaluate recorded tests before approving any live endpoint

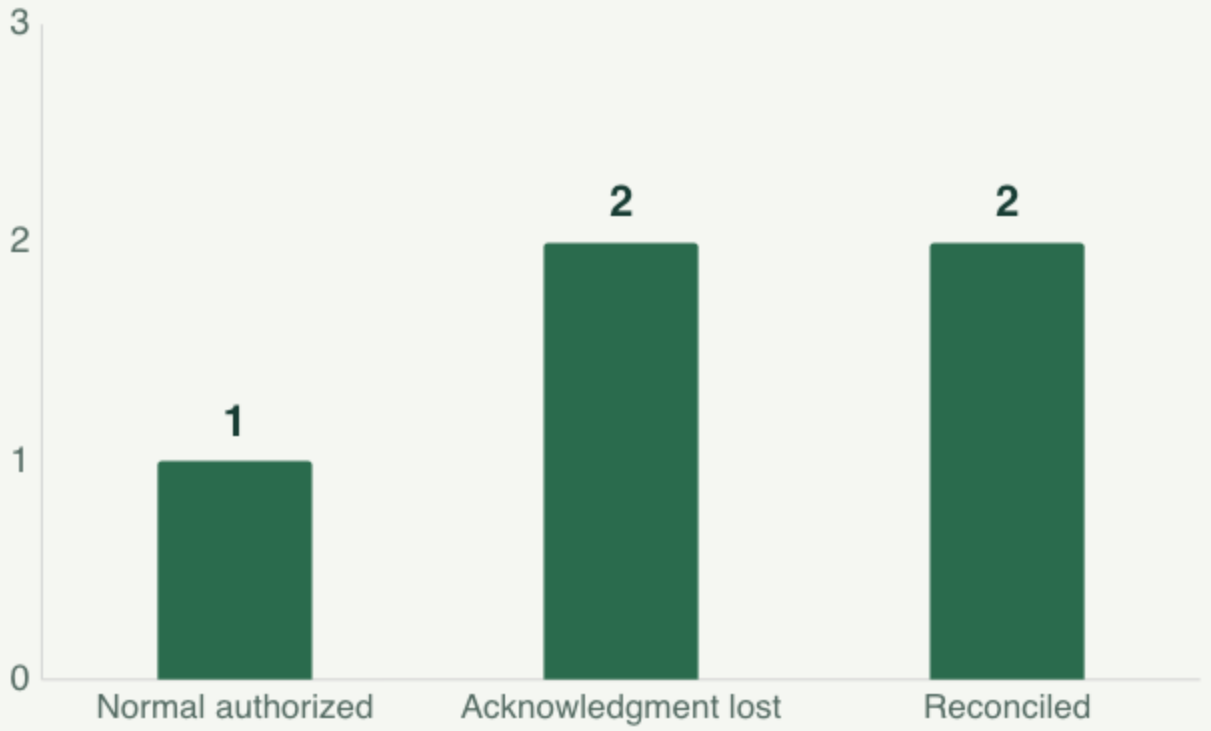
Acceptance authorizes evaluation. Production payment approval remains a separate decision.

Authorization evidence through the demo

2 requests

2 issuer receipts

Reconciliation resolves uncertainty
without another authorization.



Observed in the synthetic guided run

Architecture and platform placement

Why build rather than buy?

The pilot evaluates investigation and policy controls around existing payment services. A replacement case and buy-versus-build assessment remain open.

Where does Databricks enter the critical path?

Start with governed history, features and evaluation. Validate tail latency and freshness before selecting online serving placement.

Why SQLite?

It makes local state transitions atomic. The pilot must select an operational store for tenancy, concurrency and external side effects.

Authorization and failure correctness

Do we guarantee exactly-once authorization?

No. We demonstrate session-local idempotency. Production needs merchant-scoped keys and explicit acceptance-versus-persistence crash tests.

How do we resolve a timed-out issuer request?

Use an authoritative provider lookup or reconciliation receipt. Without trustworthy evidence, preserve UNKNOWN and prevent blind retry.

Can an analyst override a decline?

No. Release applies only to REVIEW before authorization. Blocks, failed authentication and issuer declines remain protected.

Security and approval controls

Is maker/checker enforcement complete?

The demo enforces distinct simulated roles. Production needs authenticated identity, trusted role assignment and server-side authorization.

Does the evidence hash prevent tampering?

No. It binds approval to the evidence used. Audit integrity needs protected storage, retention and an agreed integrity mechanism.

Are we PCI compliant?

We make no certification claim. The demo uses synthetic tokens. Real-card integration needs a separate scope and security assessment.

Risk quality and operating behavior

What is the latency SLA?

Unestablished. Owners must allocate the response budget, then measure tail latency and timeout behavior under representative load.

What happens when risk evaluation is unavailable?

The demo holds the request. Production fallback must be an approved risk policy with bounded timeouts and observable outcomes.

How accurate are the fraud rules?

Accuracy is unmeasured. Evaluate representative labeled data and the cost of false declines before making detection claims.

Acceptance and interpretation

What does a green graph mean?

The workflow reached safely handled demo outcomes, including blocks or declines. Green does not certify a legitimate payment.

What should the manager approve today?

The bounded pilot scope, accountable owners and acceptance conditions. Live endpoints and production rollout need later approval.

What if the prototype has not proved a property?

We have not validated that yet. Make it an explicit pilot gate and return with measured evidence.